GUÍA OFICIAL DE DESARROLLO FRONTEND — JOSBECORSA

Manual de Desarrollo Frontend

Guía Integral de Arquitectura UI con JavaScript (ES6+), React y TailwindCSS para el Sistema Mayorista BeaverVault

Proyecto:	BeaverVaultFrontend
Lenguaje Obligatorio:	**JavaScript puro (ES6+) con JSX** (No TypeScript)
Framework UI:	React 18 / 19 + Vite + React Router DOM v6+
Estilos y Diseño:	TailwindCSS + Lucide React (Diseño limpio y responsivo)
Cliente HTTP:	Axios con Interceptores de Token Bearer JWT
Backend Conectado:	API REST Node.js/Express (http://localhost:3000/api)
Líder de Proyecto:	Josue — Equipo JOSBECORSA
Documentación API:	Scalar UI en vivo en http://localhost:3000/docs

REGLA DE ORO OPERATIVA DE JOSBECORSA:
¡NO HAY MERMA! En la transformación de jabas a kilogramos, el peso bruto de la jaba se convierte al 100% en kilogramos disponibles en stock en la tabla inventarios_kilos. Ni el backend ni el frontend deben aplicar porcentajes de deducción o factores de pérdida. El negocio mayorista comercializa íntegramente el peso recibido en balanza.

Propósito del Documento: Este manual proporciona al equipo de frontend todas las especificaciones, estructuras de carpetas, flujos de autenticación, componentes clave en JavaScript y pautas para consumir la API de JOSBECORSA asegurando una interfaz ágil, reactiva y 100% fiel a las 4 etapas del negocio.

1. Estructura de Carpetas del Proyecto (JavaScript + React)

El proyecto debe organizarse bajo una arquitectura modular y desacoplada utilizando **JavaScript (ES6+)** con componentes .jsx y utilidades .js:

Carpeta / Archivo	Tecnología	Propósito y Responsabilidad
`src/services/api.js`	JavaScript / Axios	Instancia de Axios con baseURL e interceptores automáticos de Bearer Token.
`src/context/AuthContext.jsx`	React Context	Estado global de autenticación: token, usuario activo, login y logout.
`src/components/common/`	React + Tailwind	Componentes reutilizables: Navbar, Sidebar, Modal, Badge, Button, Spinner, Table.
`src/components/forms/`	React + Tailwind	Inputs controlados y selector en cascada para la Matriz Estricta de Palta.
`src/pages/acopio/`	React JSX	Vistas de Acopio (Provincia): pesaje de jabas y gastos de seleccionadores.
`src/pages/transporte/`	React JSX	Vistas de Transporte: asignación de camión, chofer, placa y costo de flete.
`src/pages/distribucion/`	React JSX	Vistas de Lima: liquidación de estibadores y traslado entre cámaras de frío.
`src/pages/ventas/`	React JSX	POS en balanza: botón **Convertir a Kilos**, venta por peso y emisión de fiados.
`src/utils/constants.js`	JavaScript	Matriz estricta de tipos de palta y calidades, enums de estados y colores.
`src/utils/formatters.js`	JavaScript	Formato de moneda (S/. 1,250.00), fechas peruanas y pesos en kilogramos.

Comandos de Inicialización del Proyecto

```
# 1. Crear el proyecto con Vite y plantilla React (JavaScript)
npm create vite@latest BeaverVaultFrontend -- --template react

# 2. Entrar a la carpeta e instalar librerias necesarias
cd BeaverVaultFrontend
npm install axios react-router-dom lucide-react

# 3. Instalar TailwindCSS y sus herramientas
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p

# 4. Iniciar servidor de desarrollo en http://localhost:5173
npm run dev
```

2. Conexión con la API y Ciclo de Autenticación

El frontend se comunica con el backend de forma segura mediante **Axios** e inyección automática del Bearer Token JWT.

Configuración del Cliente HTTP (src/services/api.js)

```
// src/services/api.js - Instancia Axios centralizada
import axios from 'axios';

const API_BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:3000/api';

export const api = axios.create({
  baseURL: API_BASE_URL,
  headers: { 'Content-Type': 'application/json' }
});

// Interceptor de Salida: Agrega el Token a cada peticion
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Interceptor de Entrada: Captura vencimiento de token (401)
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

Contexto de Autenticación (src/context/AuthContext.jsx)

```
// src/context/AuthContext.jsx - Estado global de sesion
import { createContext, useContext, useState, useEffect } from 'react';
import { api } from '../services/api';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => JSON.parse(localStorage.getItem('user')) || null);
  const [token, setToken] = useState(() => localStorage.getItem('token') || null);

  const login = async (correo, contrasena) => {
    const { data } = await api.post('/auth/login', { correo, contrasena });
    setToken(data.token);
    setUser(data.superAdmin);
    localStorage.setItem('token', data.token);
    localStorage.setItem('user', JSON.stringify(data.superAdmin));
    return data;
  };

  const logout = () => {
    setToken(null);
    setUser(null);
    localStorage.clear();
    window.location.href = '/login';
  };

  return (

      {children}

  );
}

export const useAuth = () => useContext(AuthContext);
```

3. Las 4 Etapas Operativas en la Interfaz de Usuario

Cada pantalla del frontend debe reflejar fielmente la operación física del negocio mayorista:

Etapa	Pantalla / Componente	Interacciones de Usuario en UI	Endpoint Asociado
1. Acopio (Provincia)	`pages/acopio/AcopioList.jsx` `AcopioCreateModal.jsx`	• Crear lote con código y fecha. • Agregar jabas con código de color y peso. • Selector en cascada Tipo → Calidad. • Modal de liquidación de seleccionadores.	`POST /api/acopios` `POST /api/acopios/:id/gastos/seleccionador`
2. Transporte (Ruta)	`pages/transporte/TransporteList.jsx` `FleteModal.jsx`	• Asignar transportista (Nombre chofer, placa). • Ingreso de costo real de flete. • Cambio de badge a **En Tránsito** (amarillo).	`POST /api/acopios/:id/transporte`
3. Lima (Distribución)	`pages/distribucion/DistribucionView.jsx`	• Recepción de camión (badge azul). • Liquidar estibadores por bulto. • Mover jabas entre Cámara de Frío y Puesto.	`POST /api/acopios/:id/gastos/descarga` `PATCH /api/jabas/:id/ubicacion`

4. Ventas (Puesto)	`pages/ventas/PuntoVentaPOS.jsx`
	`TransformarKilosModal.jsx`
		• **Botón 'Convertir a Kilos' (¡Sin Merma!)**.
		• Venta por balanza en kilos y precio.
		• Registro de venta o crédito (Fiado).
			`POST /api/jabas/:id/transformar-kilos`
			`POST /api/ventas`
			`POST /api/fiados`

Matriz de Palta en JavaScript (src/utils/constants.js)

```
// src/utils/constants.js - Matriz estricta de validacion
export const MATRIZ_CLASIFICACION = {
  Fuerte: ['1era', '2da', '3era', '4ta', 'Semimanchada', 'Manchada'],
  Hass: ['1era', '2da', '3era', '4ta', 'Manchada'],
  Naval: ['Limpia', 'Manchada'],
  Villacampa: ['1era', 'Manchada'],
};

export const TIPOS_PALTA = Object.keys(MATRIZ_CLASIFICACION);
export const COLORES_JABA = ['Rojo', 'Azul', 'Verde', 'Amarillo', 'Negro', 'Blanco', 'Naranja'];
export const UBICACIONES = ['Acopio', 'Camara Frio', 'Puesto Principal', 'Puesto Contingencia'];
```

4. Componentes Clave en React (JavaScript + Tailwind)

A continuación se detallan los dos componentes de mayor importancia operativa en la interfaz:

A. Selector en Cascada de Palta (CascadeSelect.jsx)

Garantiza que el usuario solo elija calidades válidas según el tipo de palta, evitando errores 400:

```
// src/components/forms/CascadeSelect.jsx
import { MATRIZ_CLASIFICACION, TIPOS_PALTA } from '../../utils/constants';

export function CascadeSelect({ tipoPalta, categoria, onChangeTipo, onChangeCategoria }) {
  const categoriasDisponibles = tipoPalta ? MATRIZ_CLASIFICACION[tipoPalta] || [] : [];

  const handleTipoChange = (e) => {
    const nuevoTipo = e.target.value;
    onChangeTipo(nuevoTipo);
    onChangeCategoria(''); // Reinicia categoria al cambiar de tipo
  };

  return (


        Tipo de Palta

          -- Seleccionar Tipo --
          {TIPOS_PALTA.map(t => {t})}



        Categoría / Calidad
         onChangeCategoria(e.target.value)}
          disabled={!tipoPalta} className='w-full border rounded-lg p-2 text-sm disabled:bg-slate-100'>
          -- Seleccionar Calidad --
          {categoriasDisponibles.map(c => {c})}



  );
}
```

B. Modal de Conversión a Kilos (TransformarKilosModal.jsx) — ¡SIN MERMA!

```
// src/pages/ventas/TransformarKilosModal.jsx
import { useState } from 'react';
import { api } from '../../services/api';

export function TransformarKilosModal({ jaba, ubicacionId, onClose, onSuccess }) {
  const [loading, setLoading] = useState(false);

  const handleTransformar = async () => {
    setLoading(true);
    try {
      await api.post(`/jabas/${jaba.id}/transformar-kilos`, { ubicacionId });
      onSuccess();
      onClose();
    } catch (err) {
      alert(err.response?.data?.error || 'Error al transformar jaba');
    } finally { setLoading(false); }
  };

  return (


        Convertir Jaba a Kilos

          La jaba {jaba.codigoColor} ({jaba.tipoPalta} - {jaba.categoria}) con un peso de
           {jaba.pesoBruto} kg pasará íntegramente al inventario de kilos comercial.


          ✓ REGLA JOSBECORSA: Sin merma. Se habilitarán exactamente {jaba.pesoBruto} kg para venta.


          Cancelar

            {loading ? 'Procesando...' : 'Confirmar Transformación'}




  );
}
```

5. Servicios API Desacoplados (src/services/)

Las llamadas a la API deben estar aisladas en archivos de servicio para no ensuciar la lógica visual:

Servicio de Acopio (src/services/acopioService.js)

```
// src/services/acopioService.js
import { api } from './api';

export const acopioService = {
  // Listar acopios de una empresa
  listarPorEmpresa: async (empresaId) => {
    const { data } = await api.get(`/empresas/${empresaId}/acopios`);
    return data;
  },

  // Registrar nuevo lote con pesaje de jabas
  crearAcopio: async (payload) => {
    const { data } = await api.post('/acopios', payload);
    return data;
  },

  // Registrar pago a seleccionadores
  registrarPagoSeleccionador: async (acopioId, { seleccionadorNombre, cantidadJabas, costoPorJaba }) => {
    const { data } = await api.post(`/acopios/${acopioId}/gastos/seleccionador`, {
      seleccionadorNombre, cantidadJabas, costoPorJaba
    });
    return data;
  }
};
```

Servicio de Ventas y Kilos (src/services/ventaService.js)

```
// src/services/ventaService.js
import { api } from './api';

export const ventaService = {
  // Consultar kilos disponibles en balanza por ubicacion
  obtenerStockKilos: async (ubicacionId) => {
    const { data } = await api.get(`/ubicaciones/${ubicacionId}/inventario-kilos`);
    return data;
  },

  // Registrar venta al por mayor por kilogramos
  registrarVenta: async ({ empresaId, ubicacionId, clienteNombre, detalles }) => {
    const { data } = await api.post('/ventas', { empresaId, ubicacionId, clienteNombre, detalles });
    return data;
  },

  // Registrar fiado (cuenta por cobrar)
  registrarFiado: async ({ empresaId, ventaId, clienteNombre, montoTotal, montoPendiente }) => {
    const { data } = await api.post('/fiados', { empresaId, ventaId, clienteNombre, montoTotal, montoPendient
e });
    return data;
  }
};
```

6. Checklist de Implementación Frontend

Hitos técnicos que el equipo frontend debe seguir para entregar una aplicación completa y de calidad:

Hito	Acción Técnica en JavaScript / React	Resultado Esperado
1. Proyecto	Inicializar con Vite + React (JavaScript) y configurar TailwindCSS.	Servidor en localhost:5173
2. Cliente API	Crear api.js con base URL e interceptores de Token y error 401.	Peticiones seguras con Bearer
3. Login	Pantalla de inicio de sesión con AuthContext y guardado de token.	Redirección al dashboard OK
4. Matriz UI	Formulario de jabas con CascadeSelect.jsx respetando tipos y calidades.	Cero errores 400 por clasificación
5. Flete y Lima	Módulos de transporte y distribución en frío con badges de estado.	Trazabilidad completa del lote
6. Sin Merma	Botón 'Convertir a Kilos': Traslado 1:1 de peso bruto a stock de balanza.	Stock en kilos 100% exacto
7. POS & Fiados	Punto de venta por kilogramos, cálculo de subtotal y registro de fiados.	Facturación y cobranzas al día

Recomendaciones de UI/UX para el Equipo

• **Formato de Moneda y Balanza:** Usar siempre formatters.js para mostrar S/. 1,450.00 y pesos con dos decimales (420.50 kg).
• **Feedback Visual:** Mostrar notificaciones tipo Toast (éxito en verde, error en rojo) en cada transacción.
• **Manejo de Errores:** Si el backend responde 400 o 409, extraer err.response?.data?.error o err.response?.data?.detalles y mostrarlo al usuario.
• **Documentación en Vivo:** Mantener abierto http://localhost:3000/docs para inspeccionar payloads.

Compromiso de Integración: Siguiendo esta guía en JavaScript puro con React y TailwindCSS, el equipo de frontend construirá una aplicación comercial robusta, rápida e intuitiva para los operadores de acopio, transportistas y vendedores del puesto mayorista de JOSBECORSA.